Features of an effective backup tool

Effective and efficient software is required for backing up data. Some of the major features a backup tool for personal as well as corporate computing must have are:

- Non-redundant data/ de-duplication
- Security with multi-layered encryption
- Dynamic compression and archiving
- Dynamic repository management
- Automatic backups
- Cloud based backup and recovery
- Import/export for multiple platforms
- Version tracking
- Configurability, with the scope for extensions using plugins
- Command line as well as GUI panels for better usability
- Transaction fault tolerance to avoid data loss
- Volume oriented to support compression, splitting and merging for multiple devices and platforms
- Malware scanning
- Universal view and updates
- Cross-platform
- Support for multiple data formats
- Support for multiple databases
- Generation of reports, alerts and logs

Free and open source Python tools

Python is a powerful programming language used for many high performance computing domains including cloud computing, parallel architectures, Big Data, data mining, machine learning, grid computing, heterogeneous databases, and many others.

It is enriched by a number of libraries to enable de-duplicating, secured and cloud based backup implementations, etc. Bakthat and Attic are very effective tools which are based on Python and can be used for a general-purpose and highly efficient backup system.

Bakthat, a local and cloud based backup tool

Bakthat is a Python based framework developed for secured and efficient backup. It is available with the command line as well as the module for Python to support cloud backups on Amazon Cloud Infrastructure like Amazon S3, Amazon Glacier as well as OpenStack Swift. This tool is able to compress, encrypt and upload documents to cloud servers. The base licence of Bakthat is MIT.

Features of Bakthat include the following:

- Dynamic compression
- Encryption using Beefish
- Uploading/downloading to Glacier or S3 with Python Boto Programming
- Maintaining the local backups in the SQLite database using Peewee
- Deletion of older files and documents as per the grandfather-father-son method
- Ability to sync the backups database between multiple clients via a centralised server
- Exclude files using *.gitignore*
- Usage of plugins and extensions for better performance

Installing Bakthat: Bakthat can be installed very easily using Pip as the Linux distribution:

```
$ pip install bakthat
```

You can auto configure Bakthat as shown below:

```
$ bakthat configure
```

Then, create a backup, as follows:

```
$ bakthat backup MyBackup
Backing up MyBackup
Password (blank to disable encryption):
Password confirmation:
Compressing...
Encrypting...
Uploading...
Upload completion: 0%
Upload completion: 100%
```

There is an alternate method too:

```
$ cd MyDirectory
$ bakthat backup
INFO: Backing up MyDirectory
```

You can show and restore the backup, as follows:

```
$ bakthat show
<TimeStamp> MyBackup.tgz.enc
$ bakthat restore mydir
```

You can also back up to Amazon Glacier (Bakthat on Amazon Cloud Infrastructure):

```
$ bakthat backup -d glacier
INFO: Backing up MyDirectory
Password (blank to disable encryption):
```

The latest version of a backup can be restored from Amazon Glacier by specifying the beginning of the file name, as shown below:

```
$ bakthat restore -f bak
INFO: Restoring MyBackUp.tgz.enc
Password:
INFO: Downloading...
INFO: Decrypting...
INFO: Uncompressing...
$ bakthat restore -f MyRemoteFile -d glacier
```